

Guía - Estructuras de Datos en C++

Una estructura de datos hace referencia a una organización específica sobre un conjunto de datos y sus operaciones asociadas. Un buen ejemplo de una estructura de datos es un arreglo en C++.

A lo largo del club veremos problemas en los cuáles el uso de una estructura de datos u otra será un factor clave a la hora de resolver el problema e implementar su solución.

Es por esto que desde el club decidimos facilitarles una guía rápida sobre algunas de las estructuras de datos que vienen incluidas en la librería STL (Standard Template Library) de C++.

A tener en cuenta

Las estructuras de datos de la librería STL que ocuparemos están definidas como plantillas, lo cual nos permite el manejo sobre distintos tipos de datos (int, string, bool, float, etc.) sin la necesidad de emplear una colección distinta para cada tipo. Esto se logra pasando el tipo que queremos utilizar como parámetro cuando declaramos un vector, pila o demás estructuras.

Para poder movernos dentro de las colecciones, a menudo emplearemos el uso de iteradores. En pocas palabras, un iterador funciona como un puntero "seguro", por lo cual no tendremos que preocuparnos por la aritmética de punteros y el manejo de memoria.

Vector

Similar a cómo funciona un arreglo, almacena datos en una memoria contigua pero con un tamaño dinámico (por lo tanto no hace falta declarar su tamaño al declarar un vector). Cuando se queda sin espacio duplica su capacidad, reubicando sus elementos en un nuevo lugar. Se puede acceder a sus elementos directamente a través de la notación de índices.

Método	Descripción	Complejidad Temporal
push_back(value)	Agrega un elemento con el valor value al final del vector.	$O(n)$
pop_back()	Elimina el elemento que se encuentra al final del vector.	$O(1)$
at(pos)	Devuelve el elemento que se encuentra en la posición pos .	$O(1)$



<code>insert(pos, value)</code>	Inserta un elemento de valor value en la posición pos .	$O(n)$
<code>erase(pos)</code>	Elimina el elemento en la posición pos .	$O(n)$
<code>clear()</code>	Elimina todos los elementos del vector.	$O(n)$
<code>size()</code>	Devuelve el tamaño actual del vector.	$O(1)$
<code>empty()</code>	Devuelve un valor booleano, true si está vacío y false si tiene por lo menos un elemento.	$O(1)$
<code>front()</code>	Devuelve el primer elemento.	$O(1)$
<code>back()</code>	Devuelve el último elemento.	$O(1)$
<code>begin()</code>	Devuelve un iterador apuntando al primer elemento.	$O(1)$
<code>end()</code>	Devuelve un iterador apuntando al último elemento.	$O(1)$

Stack/Pila

Es una estructura con filosofía LIFO (Last In, First Out), es decir, el último elemento agregado es el primero en ser retirado.

Método	Descripción	Complejidad Temporal
<code>push(value)</code>	Agrega un elemento con el valor value al inicio de la pila.	$O(1)$
<code>pop()</code>	Elimina el elemento en el fondo de la pila.	$O(1)$
<code>top()</code>	Devuelve el primer elemento.	$O(1)$
<code>size()</code>	Devuelve la cantidad de elementos en la pila.	$O(1)$



empty()	Devuelve un valor booleano, true si está vacío y false si tiene por lo menos un elemento.	0(1)
---------	---	------

Queue/Fila

Es una estructura con filosofía FIFO (First In, First Out), es decir, los elementos se insertan al final y se eliminan desde el inicio.

Método	Descripción	Complejidad Temporal
push(value)	Agrega un elemento con el valor value al final de la fila.	0(1)
pop()	Elimina el elemento que se encuentra al inicio de la fila.	0(1)
front()	Devuelve el elemento que está al principio de la fila.	0(1)
back()	Devuelve el elemento que está al final de la fila.	0(1)
size()	Devuelve la cantidad de elementos en la fila.	0(1)
empty()	Devuelve un valor booleano, true si está vacío y false si tiene por lo menos un elemento.	0(1)

Map

Actúa como un diccionario ordenado. Cada clave es única y los elementos se almacenan ordenados por clave. En caso de querer utilizar un **map** que permita el duplicado de claves, ver [multimap](#). Es importante tener en cuenta que en **map** se ordena el contenedor según las claves del contenedor. En caso de querer utilizar su versión no-ordenada, ver [unordered_map](#). Notar que también existe una versión desordenada para **multimap**.



Método	Descripción	Complejidad Temporal
<code>insert(pair_value)</code>	Inserta un par clave-valor en el contenedor.	$O(\log(n))$
<code>operador[key]</code>	Asigna un elemento con la clave key o accede al elemento con esa clave.	$O(\log(n))$
<code>at(key)</code>	Devuelve el elemento con la clave key . Si no existe, entonces devuelve un error.	$O(\log(n))$
<code>erase(iter_pos)</code>	Elimina el elemento que se encuentra en la posición del iterador iter_pos .	$O(1)$
<code>erase(key)</code>	Elimina el elemento con la clave key .	$O(\log(n))$
<code>size()</code>	Devuelve la cantidad de elementos en el contenedor.	$O(1)$
<code>empty()</code>	Devuelve un valor booleano, true si está vacío y false si tiene por lo menos un elemento.	$O(1)$
<code>clear()</code>	Elimina todos los elementos del contenedor.	$O(n)$
<code>find(key)</code>	Devuelve un iterador que apunta al elemento con la clave key .	$O(\log(n))$
<code>count(key)</code>	Devuelve la cantidad de elementos con la clave key .	$O(\log(n))$
<code>begin()</code>	Devuelve un iterador apuntando al primer elemento.	$O(1)$
<code>end()</code>	Devuelve un iterador apuntando al último elemento.	$O(1)$



Set/Conjunto

Es un conjunto ordenado que no permite valores duplicados. En C++ también se encuentra su versión que permite duplicados, ver [multiset](#). Al ser un conjunto ordenado, esto afecta a la eficiencia de sus métodos. Existe una alternativa no-ordenada con los mismos métodos listados, ver [unordered_set](#).

Método	Descripción	Complejidad Temporal
<code>insert(value)</code>	Agrega value al conjunto.	$O(\log(n))$
<code>erase(iter_pos)</code>	Elimina el elemento que se encuentra en la posición del iterador iter_pos .	$O(1)$
<code>erase(value)</code>	Elimina el elemento con el valor value .	$O(\log(n))$
<code>size()</code>	Devuelve la cantidad de elementos en el conjunto.	$O(1)$
<code>empty()</code>	Devuelve un valor booleano, true si está vacío y false si tiene por lo menos un elemento.	$O(1)$
<code>clear()</code>	Elimina todos los elementos del contenedor.	$O(n)$
<code>find(value)</code>	Devuelve un iterador que apunta al elemento con el valor buscado.	$O(\log(n))$
<code>count(value)</code>	Devuelve la cantidad de elementos con el valor value .	$O(\log(n))$
<code>begin()</code>	Devuelve un iterador apuntando al primer elemento.	$O(1)$
<code>end()</code>	Devuelve un iterador apuntando al último elemento.	$O(1)$



Ejemplos de Código

```
ejemplo_vector.cpp

#include <bits/stdc++.h>
using namespace std;

int main() {
    // Declaración e inicialización
    vector<int> vec = {73, 11, 13};

    // Agregar elementos
    vec.push_back(1); // Agrega 4 al final
    vec.push_back(41); // Agrega 5 al final

    // Eliminar elementos
    vec.pop_back(); // Elimina el último (41)

    // Ordena el vector de menor a mayor
    sort(vec.begin(), vec.end());

    cout << "El elemento en el índice 2 es: " << vec[2] << "\n";

    // Iterar sobre el vector
    cout << "Vector: ";
    for (int val : vec) {
        cout << val << " ";
    }

    return 0;
}
```



ejemplo_pila.cpp

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> pila;

    // Agrego elementos en la cima de la pila
    pila.push(1);
    pila.push(2);
    pila.push(3);

    // Creo una copia de una pila
    stack<int> copia(pila);

    // Muestro los elementos dentro de la pila
    // Al finalizar, la pila quedará vacía
    while (!pila.empty()) {
        cout << pila.top() << " ";
        pila.pop();
    }

    return 0;
}
```



ejemplo_fila.cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Declaro una fila de enteros q
    queue<int> q;

    // Inserto 3, 4 y 5 en la fila q
    q.push(3);
    q.push(4);
    q.push(5);

    // Creo una copia a partir de q
    queue<int> temp(q);

    // Muestro los elementos de q utilizando
    // su copia
    while(!temp.empty()) {
        cout << temp.front() << " ";
        temp.pop();
    }
    return 0;
}
```



ejemplo_set.cpp

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    // Creo un set de enteros
    set<int> s = {9, 3, 5, 0, 2, 1};

    // Agrego elementos al set
    s.insert(10);
    s.insert(5);

    // Accedo al primer elemento
    auto it1 = s.begin();
    // Accedo al tercer elemento
    auto it2 = next(it1, 2);
    cout << *it1 << " " << *it2 << "\n";

    // Elimino el primer elemento utilizando
    // un iterador
    s.erase(s.begin());

    // Busco el elemento con valor igual a 3
    auto it = s.find(0);
    if (it != s.end()) cout << *it;
    else cout << "Elemento No Encontrado!\n";

    // Itero sobre el conjunto usando iteradores
    for(auto it = s.begin(); it != s.end(); it++)
        cout << *it << " ";

    return 0;
}
```

